

The Geometry of Attention: Decoding SOTA Programming Models Through Opus 4.5's Uniform Attention Distribution

I recently discovered that when asking Claude Opus 4.5 a series of questions, it would not only solve the new problem but also re-answer the entire sequence of previously resolved questions. This perfectly corroborates my earlier hypothesis: for a SOTA model in the programming domain like Opus 4.5, its attention can still be distributed evenly across the entire long-context window. This specific distribution of attention explains why Opus 4.5 possesses such formidable programming capabilities—since programming, by its nature, requires an exceptionally uniform focus across various parts of the codebase.

I have previously visualized the varying weights of attention distribution for small models across different domains. It is evident that in the programming domain, attention is significantly more balanced. In contrast, in open-ended domains, the attention tends to be far more concentrated on the most recent context. For open-ended tasks, the relative attention metric of the model's second Chain-of-Thought (CoT) step toward its first step is approximately 0.55. However, on programming datasets like LeetCode, this figure drops to less than 0.4.

For programming problems, the model must distribute its attention across various parts of the context; it doesn't need to "obsess" over its own previously generated CoT steps as much as it does in open-ended questions. The SOTA performance of Opus 4.5 in programming is no coincidence. It is so pronounced that even when I asked a standard new question, it felt compelled to re-address my previous queries in one go, reflecting its powerful global context processing.

Much like the logic of Reinforcement Learning, sometimes we explore hundreds of steps without visible progress. But after synthesizing these steps—much like a model aggregating its long-context attention—we often reach a breakthrough where the logic suddenly converges and clarifies.

